

A two-stage method for doubly resource constrained elementary shortest path problems

Xuefeng Jin, Shunzheng Yu*

School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou 510006, PR China

ARTICLE INFO

Keywords:

Constrained shortest paths
Shortest path problems
Double sided constraints
Preprocessing
Pulse algorithm

ABSTRACT

The resource constrained shortest path problem (RCSP) has been extensively studied. However, its more general version, the doubly resource constrained (elementary) shortest path problem (DRCESPP), is usually not concerned and no exact solution algorithm has been found in the literature. In this work, an exact two-stage method is proposed for solving the DRCESPP. For leveraging resource lower limits, an estimation to the maximum resource consumption of all elementary source–destination paths is proposed in the first stage. This makes the network be reduced more significantly and the algorithm be terminated in the first stage. By introducing an effective search direction into the conventional Pulse algorithm, a LB-first Pulse algorithm is proposed in the second stage to find the exact solution from the reduced network output in the first stage. Extensive computational experiments on various test instances are conducted. The numerical results show that the maximum resource consumption estimation method and the improved Pulse algorithm are effective, and the proposed two-stage method outperforms the Pulse algorithm in solving complex instances.

1. Introduction

As an extension of the well-known shortest path problem (SPP) in the field of combinatorial optimization, the resource constrained shortest path problem (RCSP) aims at finding a minimum cost path between two nodes, subject to the constraint that the resource consumption along the path cannot exceed upper limits. Distance, cost, capacity, time, energy, delay, etc. are typical resources that need to be constrained. Further, when each resource is constrained with a double sided inequality (both upper and lower limits), the doubly resource constrained shortest path problem (DRCESPP) arises, which can be applied in solving equality constrained knapsack problems [1] and pricing and billing for differentiated QoS guarantees in intelligent scenarios such as computing power network, remote autonomous driving and Cloud VR. Besides, the DRCSPP commonly arises in the traffic balancing problem in telecommunication networks. In this paper, we aim to propose exact algorithms to solve the DRCSPP.

Formally, consider a directed cyclic graph $G = (V, A)$, where V is the set of n nodes and A is the set of m directed arcs (edges). K nonnegative features are associated with each arc $(i, j) \in A$, the first of which is the arc cost, and the rest are $K - 1$ different resource consumptions along the arc. Let w_{ij}^k represent the k th scalar feature of arc (i, j) . A path $\pi_{uw} = (v_0 = u, v_1, \dots, v_p = w)$ from node u to w is a sequence of nodes such that $(v_{i-1}, v_i) \in A$ holds for all $i \in \{1, \dots, p\}$. Let $W^k(\pi) = \sum_{(i,j) \in A(\pi)} w_{ij}^k$ be the total quantity of the k th weight feature

(cost or resource) accumulated along the path π , where $A(\pi)$ denotes the set of all arcs on π . The Doubly Resource Constrained Elementary Shortest Path Problem (DRCESPP) is to find a least cost elementary path (no node is repeated) from a source node s to a destination node t such that the cumulative quantity of each resource consumed along the path belongs to a specific interval (a double sided inequality), which can be formulated as a mixed integer linear programming (MILP) as follows:

$$\min \sum_{(i,j) \in A} w_{ij}^1 x_{ij} \quad (1)$$

s.t.

$$\sum_{(i,j) \in A} x_{ij} - \sum_{(j,i) \in A} x_{ji} = \begin{cases} 1 & \text{if } i = s, \\ 0 & \text{if } i \in V \setminus \{s, t\}, \forall i \in V, \\ -1 & \text{if } i = t, \end{cases} \quad (2)$$

$$L_k \leq \sum_{(i,j) \in A} w_{ij}^k x_{ij} \leq U_k, \quad k = 2, \dots, K, \quad (3)$$

$$w_{ij}^k \geq 0, \quad k = 1, \dots, K, \quad (4)$$

$$u_i - u_j + n x_{ij} \leq n - 1, \quad \forall (i, j) \in A, \quad (5)$$

$$x_{ij} \in \{0, 1\}, \quad u_i \in \mathbb{R}, \quad (6)$$

* Corresponding author.

E-mail addresses: jinx@mails.ccnu.edu.cn (X. Jin), syu@mail.sysu.edu.cn (S. Yu).

where the zero-one decision variable x_{ij} denotes whether the optimal path contains arc (i, j) . L_k and U_k represent the lower and upper limits on the $(k-1)$ st resource consumed on the path respectively and $0 \leq L_k \leq U_k$. The shortest path problem can be formulated by (1), (2), and (6). The addition of double sided constraints (3) makes the SPP a DRCSPP. Further, subtour elimination constraints (5) guarantee that the optimal path is elementary, and accordingly the problem is transformed into the DRCESPP.

The RCSPP is known to be NP-hard even if the graph is acyclic and only single one-sided resource constraint is considered (cf. [2]). While the RCESPP has also proved to be NP-hard in the strong sense [3] and the DRC(E)SPP is generally much harder than the RCSPP [4].

Challenges in solving the DRC(E)SPP have not been addressed in the literature. The following challenges exist: (1) The dominance mechanism that is applied to filter dominated paths in many classic algorithms, such as labeling algorithms and the Pulse algorithm, will not work when lower limits are added because the non-dominated paths may not satisfy the lower limits while the dominated one does. This results in larger search space to be explored, more labels to be extended, and longer time to be taken in solving the DRCSPP. (2) Because the longest elementary path cannot be computed in an efficient way in a graph containing positive cost cycles (the corresponding problem is NP-hard), the maximum length of a path cannot be calculated in advance and be used to check if it will reach the lower limits. Therefore, estimation methods of longest elementary path about resource features need to be explored as specific techniques before the resource lower limits can be used to simplify the graph and prune the search space. (3) Additional constraints, i.e., Eq. (5), must be imposed to ensure the elementarity since the non-redundant lower limits make the graph not g-elementary [5] (“g-elementary” means the optimal path can be naturally guaranteed to be elementary without constraints of Eq. (5)). The indispensable elementarity constraints make the problem more difficult to solve, due to the resulting strong NP-hard property. (4) In some cases, the resource upper and lower limits are tight and large, which extremely enlarges the search space and makes the existing algorithms difficult to solve.

The aforementioned challenges force the need for efficient algorithms to solve the problem with double sided constraints. To solve the elementary shortest path problem with multiple double sided resource constraints (DRCESPP), i.e., the general and difficult version of the RCSPP, we propose an exact two-stage method. Stage 1 preprocessing with a maximum resource consumption estimation method aims to reduce the network size and provide initial cost upper bounds for stage 2. In stage 2, an improved LB-first Pulse algorithm is proposed to solve the reduced network if the problem is not solved in stage 1.

The main contribution of this work includes:

- A method is proposed to estimate the maximum resource consumption, which will be exploited for network reductions and fathoming tests in the preprocessing.
- The Pulse algorithm is improved by using a more effective search direction based on the lower-bound-first search strategy instead of the original uninformed depth-first search.
- An exact two-stage method with efficient techniques is proposed for solving the DRCESPP.

The rest of the paper is organized as follows. Section 2 surveys the related literature. Section 3 details our maximum estimation methods about resource consumption, the improved Pulse algorithm, and the two-stage method and its components. Experimental results are presented and analyzed in Section 4. Section 5 concludes the paper and discusses future research.

2. Literature review

The exact solution algorithms for RCSPPs have been well studied, and new or improved algorithms are constantly being proposed. However, according to the No Free Lunch Theorem [6,7], there is no one algorithm that can be perfectly adapted to different problem forms and application scenarios. Exact solution strategies for the RCSPP can be classified into three main categories: Lagrangian relaxation [5,8–12], dynamic programming-based (DP) labeling algorithms [13–18] and Pulse algorithm [19–22].

In the general framework based on the Lagrangian method, the resource constraints are relaxed by constructing the Lagrangian function, and then the lower and upper bounds of the primal problem can be obtained by solving the Lagrangian dual problem, and finally eliminate the duality gap between the lower bound and upper bound through the gap closing procedure. Handler and Zang [8] pioneered this method, specifically, they used Kelley’s cutting plane method [23] to solve the Lagrangian dual problem exactly, and used the K shortest paths (KSP) enumeration [24] with respect to the reduced cost (the Lagrangian length) to close the duality gap. Beasley and Christofides [5] used a subgradient method to approximately solve the dual problem, and used a tree search procedure (the branch-and-bound approach) to close the gap. Mehlhorn and Ziegelmann [9] presented the hull approach that yielded better upper and lower bounds than the subgradient method used in [5] and used a dynamic programming method to close the gap. In the work of Carlyle et al. [10], the dual problem is solved by bisection search [25] and the gap is closed by their near-shortest-paths (NSP) algorithm [26].

It has been recognized that preprocessing algorithms can significantly reduce network size (i.e., eliminate nodes and edges that cannot possibly lie on any feasible or optimal path) for problems on large-scale networks. Aneja et al. [13] first proposed an iterated preprocessing procedure, and their reduction algorithm only takes into account the resource upper limits, which means that a node or arc will be deleted if the minimum consumption of at least one resource through it is still greater than the corresponding upper limit. The preprocessing procedure proposed by Dumitrescu and Boland [14] is more effective and takes advantage of not only the resource limits, but also the upper bound on cost (the best feasible solution obtained during the running of preprocessing). That is, a node or arc will also be deleted if the minimum cost through it exceeds (or equals) the current upper bound on cost. The cost bound can also be the upper bound derived from solving the Lagrangian dual problem, and then reductions can be performed on reduced costs with respect to the Lagrange multipliers, which is first introduced in [5] and used in [9,11]. Based on the fact that a network reduction may change the Lagrangian dual solutions, Muhandirame and Boland [11] first proposed an integrated algorithm that iterates between network reductions and simultaneous solution of the Lagrangian dual problems. It is worth mentioning that Carlyle et al. [10] added aggregate constraints to the preprocessing procedure presented in [13] to mitigate the conflicts between resource constraints that prevent further reductions for the case of multiple constraints.

DP methods, also known as label-setting or label-correcting algorithms, aim to find all efficient (non-dominated) labels on every node through label expansion in the form of dynamic programming. Aneja et al. [13] used a label setting algorithm in the reduced network after their preprocessing procedure. Dumitrescu and Boland [14] proposed a modified label-setting algorithm taking full advantage of information found in preprocessing and Lagrangian relaxation. Note that a standard labeling method can be improved by using the cost bound to avoid propagating unnecessary labels that cannot yield a better solution. The bound can be obtained from labeling algorithms itself [13], a preprocessing procedure [14] or the Lagrangian dual solution process [9,14].

In 2013, an elegant algorithm named Pulse algorithm was proposed by Lozano and Medaglia [19] for the RCSPP. In essence, it is a path enumeration algorithm with a depth-first search (DFS) exploration scheme

and efficient pruning strategies to discard unpromising regions of the search space are crucial to its practical performance. Computational results show that the Pulse algorithm outperforms the improved path ranking method by Santos et al. [27], the modified label-setting algorithm by Dumitrescu and Boland [14] and the three-stage approach by Zhu and Wilhelm [18]. More recently, a bidirectional search strategy, proposed by Cabrera et al. [21], was adopted to improve the Pulse algorithm by updating (decreasing) the cost bound as early as possible.

In addition to the above methods, path ranking methods [8,27,28] and A^* algorithms [29,30] are also explored for the RCSPP.

For addressing the elementary path constraint stemming from negative cost cycles or lower limits on resource consumption, an intuitive idea introduced by Beasley and Christofides [5] is to attach a dummy resource consumption of 1 to each node, and correspondingly add a constraint that the resource consumption along a path cannot exceed 1. Generally implemented by labeling algorithms, this idea is developed in [15,17,31,32] for problems with negative cost cycles such as the subproblem for the vehicle routing problem with time windows (VRPTW). For the same problem, the Pulse algorithm is also modified by Lozano et al. [20] and Li and Han [22] to satisfy the elementary path restriction.

Only a small amount of work, such as [4,5], involves double sided resource constraints. Ribeiro and Minoux [4] proposed a heuristic approach to solve the problems with a double sided inequality in sequential graphs, and no exact solution algorithm for the DRCSPP has been found in the literature.

3. Algorithms

In this section, we present our two-stage algorithm for the DRCSPP. The first preprocessing stage is an iterative procedure to eliminate unnecessary nodes and arcs and reduce the network size. The second exact searching stage uses an improved Pulse algorithm to solve the reduced network. Because the optimal solution may be found in the preprocessing stage, the exact searching stage may become unnecessary.

Now we introduce some additional notations for the preprocessing algorithm and the improved Pulse algorithm. When the k th feature is considered (the corresponding k th feature network is denoted as G_k), we denote the shortest path of the k th feature from the source node s to the node i and the shortest path from i to the destination node t by $\bar{\pi}_i^k$ and $\bar{\pi}_t^k$, respectively. Concatenate $\bar{\pi}_i^k$ and $\bar{\pi}_t^k$, and let $\bar{\pi}_i^k$ and $\bar{\pi}_{ij}^k$ be the concatenated s - t path on node i and arc (i, j) on G_k respectively, where $(i, j) \in A$. Then, total cost or resource consumption accumulated along the concatenated s - t path $\bar{\pi}_i^k$ and $\bar{\pi}_{ij}^k$ can be calculated by $W^r(\bar{\pi}_i^k) = W^r(\bar{\pi}_i^k) + W^r(\bar{\pi}_{ij}^k)$ and $W^r(\bar{\pi}_{ij}^k) = W^r(\bar{\pi}_i^k) + w_{ij}^r + W^r(\bar{\pi}_t^k)$, where $r = 1, \dots, K$. We can find that $\bar{\pi}_i^k$ and $\bar{\pi}_{ij}^k$ represent the shortest s - t path through node i and arc (i, j) respectively, and $\bar{\pi}_i^k$ represents the shortest s - t path on G_k .

Note that to get $\bar{\pi}_i^k$, we can reverse the direction of all arcs in G_k and get its reverse network G'_k , and then perform an ordinary (unconstrained) one-to-all shortest path algorithm (e.g., Dijkstra's algorithm) starting from node t on G'_k to get the shortest path to node i , and finally reverse the node sequence of the path. This can be summarized as reverse shortest path tree calculations. Besides, the faster algorithm, that is, the distance-bounded version of the Dijkstra's algorithm introduced by Ahmadi et al. [30] is adopted whenever possible.

3.1. Preprocessing algorithm

Our preprocessing procedure for the resource upper limits is similar to that of Dumitrescu and Boland [14]. However, their algorithm is not capable of dealing with resource lower limits, and our preprocessing is algorithmically more powerful and incorporates reduction and termination criteria for the lower limits and elementary path constraints to solve the DRCSPP. Specifically, our preprocessing consists

Algorithm 1: Preprocessing algorithm.

Input: G, V, A, L_k, U_k, s, t , and feature matrix w_{ij}^k
Output: cost upper bound UB , the path π_* w.r.t. UB

```

1: Init:  $UB \leftarrow \inf, \pi_* \leftarrow ()$ ;
2: delete the incoming arcs of node  $s$  and the outgoing arcs of node  $t$ ;
   // process path  $(s, t)$  separately
3: if  $(s, t) \in A$  then
4:   if  $L_k \leq w_{st}^k \leq U_k$  for all  $k \in \{2, 3, \dots, K\}$  then
5:      $UB \leftarrow w_{st}^1, \pi_* \leftarrow (s, t)$ ;
6:   end if
7:   delete arc  $(s, t)$ ;
8: end if

9: repeat                                     ▷ the main body
10:  Reductions by elementarity;
11:  for  $k = 1, \dots, K'$  do
12:    if  $k == 1$  then
13:      Tests by optimality on cost bounds;
14:      Reductions by optimality on cost bounds;
15:    else
16:      Tests by infeasibility on resource upper limits;
17:      Tests by infeasibility on resource lower limits;
18:      Reductions by infeasibility on upper limits;
19:      Reductions by infeasibility on lower limits;
20:    end if
21:  end for
22: until (no nodes are deleted, no arcs are deleted, and  $UB$  is not
   modified) or  $(|V| == 2)$  or  $(|A| == 0)$  or the maximum number of
   iterations is reached
23: Final tests by infeasibility on resource lower limits;
```

of several reduction strategies and global termination tests: reductions by elementarity, reductions/tests by optimality, and reductions/tests by infeasibility. In these mechanisms, reduction strategies are used to eliminate nodes and arcs. And any termination test taking effect means that the preprocessing can be terminated, and the problem is solved to optimality because there is certainly no better solution (fathomed from tests by optimality) or feasible solution (fathomed from tests by infeasibility) in the current reduced network.

The main body of our preprocessing algorithm is given in Algorithm 1, which is iterated until falling into a deadlock (nothing is deleted or modified) or reaching the maximum number of iterations. For the sake of simplicity, we still use G to denote the reduced network during the iterations, and V and A represent the set of its nodes and arcs.

A detailed explanation of the mechanisms and the corresponding pseudocode are given below.

3.1.1. Reductions by elementarity

If node i has only one outgoing arc and one incoming arc and they point to the same node, i.e., the tail of the incoming arc and the head of the outgoing arc are the same node (assuming j), i can be safely deleted since the path through it will contain the repeated node j and not be elementary. This reduction strategy is iteratively executed until no more nodes are deleted.

3.1.2. Reductions/tests by optimality on cost bounds

As the preprocessing procedure runs, many feasible paths will be generated from $\bar{\pi}_i^k, \bar{\pi}_{ij}^k$ and $\bar{\pi}_t^k$, and the objective function value of the optimal feasible path is denoted as UB^1 or UB , which is the best known upper bound of problem (1). As mentioned, $\bar{\pi}_i^1$ and $\bar{\pi}_{ij}^1$ represent the shortest (minimum cost) s - t path concatenated through node i and arc (i, j) , respectively, and $\bar{\pi}_i^1$ represents the shortest s - t path on the cost feature network G_1 . This information on G_1 can be exploited for

Algorithm 2: Reductions and tests by optimality on cost bounds.

```

1: calculate the shortest path trees  $\bar{\pi}_i^1$  by performing Dijkstra's
   algorithm starting from  $s$  on  $G_1$ ;
   // Tests by optimality
2: if  $W^1(\bar{\pi}_i^1) \geq UB$  then
3:   Stop ( $\pi_*$  is the optimal path);
4: else if  $\bar{\pi}_i^1$  is feasible then
5:    $UB \leftarrow W^1(\bar{\pi}_i^1)$ ,  $\pi_* \leftarrow \bar{\pi}_i^1$ ;
6:   Stop ( $\pi_*$  is the optimal path);
7: end if

8: calculate the reverse shortest path trees  $\bar{\pi}_i^1$  by performing Dijkstra's
   algorithm starting from  $t$  on  $G_1$ ;
   // Reductions by optimality through nodes
9:  $V' \leftarrow \{\}$ ;
10: for  $i \in V \setminus \{s, t\}$  do
11:   if  $W^1(\bar{\pi}_i^1) \geq UB$  then
12:     delete node  $i$ ;
13:   else if  $\bar{\pi}_i^1$  is feasible then
14:      $UB \leftarrow W^1(\bar{\pi}_i^1)$ ,  $\pi_* \leftarrow \bar{\pi}_i^1$ ; ▷ improve UB
15:     delete node  $i$ ;
16:   else
17:     add  $(i, W^1(\bar{\pi}_i^1))$  to  $V'$ ;
18:   end if
19: end for
20: for  $(i, W^1(\bar{\pi}_i^1)) \in V'$  do
21:   if  $W^1(\bar{\pi}_i^1) \geq UB$  then
22:     delete node  $i$ ;
23:   end if
24: end for

   // Reductions by optimality through arcs
25:  $A' \leftarrow \{\}$ ;
26: for  $(i, j) \in A$  do
27:   if  $W^1(\bar{\pi}_{ij}^1) \geq UB$  then
28:     delete arc  $(i, j)$ ;
29:   else if  $\bar{\pi}_{ij}^1$  is feasible then
30:      $UB \leftarrow W^1(\bar{\pi}_{ij}^1)$ ,  $\pi_* \leftarrow \bar{\pi}_{ij}^1$ ; ▷ improve UB
31:     delete arc  $(i, j)$ ;
32:   else
33:     add  $((i, j), W^1(\bar{\pi}_{ij}^1))$  to  $A'$ ;
34:   end if
35: end for
36: for  $((i, j), W^1(\bar{\pi}_{ij}^1)) \in A'$  do
37:   if  $W^1(\bar{\pi}_{ij}^1) \geq UB$  then
38:     delete arc  $(i, j)$ ;
39:   end if
40: end for

```

preprocessing. We can safely delete intermediate node i or arc (i, j) that cannot possibly lie on an optimal path (more precisely, a better path) if $W^1(\bar{\pi}_i^1) \geq UB$ or $W^1(\bar{\pi}_{ij}^1) \geq UB$. Besides, the intermediate node i will be deleted if $\bar{\pi}_i^1$ is a feasible (elementary and resource feasible) path, and the arc can be deleted in the same way, since we can conclude that $\bar{\pi}_i^1$ or $\bar{\pi}_{ij}^1$ is the optimal (feasible) path through the node or the arc in the current reduced network, and i and (i, j) are no longer valuable. Furthermore, the fathoming test will terminate the algorithm with UB as the optimal objective value if $W^1(\bar{\pi}_i^1) \geq UB$, since we fathom that the ideal optimal path $\bar{\pi}_i^1$ of the reduced network is still worse than the known optimal path w.r.t. UB . In another case, if $\bar{\pi}_i^1$ is better than the known optimal path (i.e., $W^1(\bar{\pi}_i^1) < UB$) and is a feasible path, $\bar{\pi}_i^1$ can be confirmed as the optimal path and the preprocessing can be terminated. The related pseudocode is given in Algorithm 2.

3.1.3. Reductions/tests by infeasibility on resource upper limits

The idea of reductions/tests on resource upper limits is similar to that on cost bounds. For resource k , $W^k(\bar{\pi}_i^k)$, $W^k(\bar{\pi}_{ij}^k)$ and $W^k(\bar{\pi}_{st}^k)$ on G_k represent the lower bounds on resource consumption of all s - t paths through node i , through arc (i, j) and on the current reduced network, respectively. Based on these lower bounds, our reductions and tests will be made if the bounds on a particular resource violate corresponding upper limit constraints. Specifically, the intermediate node i will be deleted if $W^k(\bar{\pi}_i^k) > U_k$, and analogously, the arc (i, j) will be deleted if $W^k(\bar{\pi}_{ij}^k) > U_k$ for some resource k , where U_k is the previously defined upper limit on the $(k - 1)$ st resource consumption. Also, the whole algorithm will be terminated by the fathoming tests if $W^k(\bar{\pi}_{st}^k) > U_k$ for some k , where $k = 2, \dots, K$.

3.1.4. Reductions/tests by infeasibility on resource lower limits

Let UB^k and UB_{ij}^k denote the maximum estimate on resource k consumption of all elementary s - t paths in the current graph and that through arc (i, j) , respectively, which means that the $(k - 1)$ st resource consumption of all elementary s - t paths is not greater than UB^k , and the consumption through (i, j) is not greater than UB_{ij}^k . Then it is evident that the preprocessing algorithm can be terminated by our fathoming tests as soon as $UB^k < L_k$ for some resource k . And the reductions by UB_{ij}^k will delete arc (i, j) if $UB_{ij}^k < L_k$, because the resource consumption through (i, j) is certainly not greater than UB_{ij}^k (thus smaller than L_k), and the path through (i, j) cannot be resource feasible.

In fact, for acyclic graphs, the longest (elementary) s - t path, i.e., the best estimate, can be computed efficiently by the dynamic programming based the topological sorting [33] with the time complexity of $O(n + m)$, but cycles are allowed in our consideration. We attempt to find the maximum estimates UB^k and UB_{ij}^k by Lemma 1 in the network containing cycles.

Lemma 1.

$$\begin{aligned}
 UB^k &= \min \left\{ \sum_{i \in V \setminus \{t\}} \max_{(i,j) \in A} w_{ij}^k, \sum_{j \in V \setminus \{s\}} \max_{(i,j) \in A} w_{ij}^k \right\}, \\
 UB_{ij}^k &= w_{ij}^k \\
 &\quad + \min \left\{ \sum_{u \in V \setminus \{i,t\}} \max_{(u,w) \in A} w_{uw}^k, \sum_{w \in V \setminus \{s,j\}} \max_{(u,w) \in A} w_{uw}^k \right\}.
 \end{aligned}$$

Proof. Let $\pi_{st} = (s, v_1, v_2, \dots, v_{n'}, t)$ be an arbitrary elementary s - t path in graph G_k , where $n' \leq n - 2$, $s \neq v_1 \neq v_2 \neq \dots \neq v_{n'} \neq t$. Let $V' = \{s, v_1, v_2, \dots, v_{n'}, t\}$ be the set of nodes on π_{st} and we have $V' \subseteq V$. The $(k - 1)$ st resource consumption accumulated along π_{st} is

$$\begin{aligned}
 W^k(\pi_{st}) &= w_{sv_1}^k + w_{v_1v_2}^k + \dots + w_{v_{n'}t}^k \\
 &\leq \max_{(s,j) \in A} w_{sj}^k + \max_{(v_1,j) \in A} w_{v_1j}^k + \dots + \max_{(v_{n'},j) \in A} w_{v_{n'}j}^k \\
 &= \sum_{i \in V' \setminus \{t\}} \max_{(i,j) \in A} w_{ij}^k \\
 &\leq \sum_{i \in V \setminus \{t\}} \max_{(i,j) \in A} w_{ij}^k.
 \end{aligned}$$

Analogously, we can get another maximum estimate by

$$\begin{aligned}
 W^k(\pi_{st}) &\leq \max_{(i,v_1) \in A} w_{iv_1}^k + \dots + \max_{(i,v_{n'}) \in A} w_{iv_{n'}}^k + \max_{(t,i) \in A} w_{ti}^k \\
 &= \sum_{j \in V' \setminus \{s\}} \max_{(i,j) \in A} w_{ij}^k \\
 &\leq \sum_{j \in V \setminus \{s\}} \max_{(i,j) \in A} w_{ij}^k.
 \end{aligned}$$

Thus, the final maximum estimate can be obtained by

$$UB^k = \min \left\{ \sum_{i \in V \setminus \{t\}} \max_{(i,j) \in A} w_{ij}^k, \sum_{j \in V \setminus \{s\}} \max_{(i,j) \in A} w_{ij}^k \right\}.$$

The maximum estimate on the resource consumption of all elementary paths through (i, j) can be proved in a similar way and omitted here. $\square$

We also note that Saigal [34] (and see [35]) presented a dynamic programming algorithm for finding the shortest path containing exactly q arcs for the one-to-all SPP, which inspires us to calculate the maximum estimate UB^k on resource consumption in another way. The maximum length of all the longest s - t paths w.r.t. different q ($q = 1, 2, \dots, n-1$) can be calculated from their q -paths algorithm with some modifications, and it is a maximum estimate rather than an exact value since the elementarity is not guaranteed by the algorithm in a graph containing cycles. Using formulas, $UB^k = \max_{q \in \{1, 2, \dots, n-1\}} W^k(\pi_{st}^{kq})$, where π_{st}^{kq} is the longest s - t path containing exactly q arcs calculated from the q -paths algorithm on G_k . This is essentially similar to the Bellman-Ford algorithm [36,37]. Although this maximum estimate is generally better than the one from Lemma 1 after the network is greatly reduced (but the opposite is the case in the early stages of iterative preprocessing), the algorithm has worse time complexity of $O((n-1)^3)$. Thus, an idea for achieving the trade-off between time overhead and preprocessing aggressiveness is to use only UB^k generated from the q -paths algorithm as a final check (the last line of the Algorithm 1). That is, only when the preprocessing w.r.t. UB^k and UB_{ij}^k generated from Lemma 1 falls into a deadlock and will be exited, the fathoming tests by infeasibility on resource lower limits w.r.t. UB^k from the q -paths algorithm will be executed only once to attempt to stop solving the problem and return an optimal path.

As an ensemble on infeasibility, the reductions/tests on resource upper limits and that on lower limits are integrated and the related pseudocode is shown in Algorithm 3. And the pseudocode of the final tests by infeasibility on lower limits is described in Algorithm 4.

To eliminate more nodes and arcs and make sharper reductions and fathoming tests, we utilize aggregate constraints proposed by Carlyle et al. [10], which can apply to the reductions and tests by infeasibility as well as the $K-1$ ordinary constraints. The c th aggregate constraint adds an additional feature (resource consumption) $w_{ij}^{K+c} = \sum_{k=2}^K \alpha_c^k w_{ij}^k$ to each arc and the corresponding lower limit $L_{K+c} = \sum_{k=2}^K \alpha_c^k L_k$ and upper limit $U_{K+c} = \sum_{k=2}^K \alpha_c^k U_k$ are associated with it, where α_c^k is the weight of the $(k-1)$ st ordinary constraint in the aggregate constraint c and $\alpha_c^k \geq 0$. For striking a balance between time and quality, we only add a single aggregate constraint based on all $\alpha_c^k = 1$, and the number of features associated with each arc will be $K' = K+1$, which corresponds to line 11 of Algorithm 1.

It can be found from Algorithms 2 and 3 that, we try to find feasible s - t paths to attempt to improve UB , thereby benefiting the reductions/tests by optimality and infeasibility in subsequent iterations.

3.2. Improved LB-first Pulse algorithm

The reduced network needs to be further solved, if the preprocessing algorithm does not find the optimal solution in the first stage. The Pulse algorithm [19] was originally proposed for the RCSPP, and showed its effectiveness because of three pruning strategies: pruning by dominance, bounds and infeasibility. While the dominance pruning strategy no longer applies to the problem with double sided constraints, thus only the bounds and infeasibility pruning strategies can be inherited from the original algorithm. Although the idea of our proposed maximum estimation methods can be used to prune unpromising search space, it incurs time overhead and is sensitive to lower limits, and thus is not considered here.

Based on the fact that smaller cost bounds UB can bring sharper pruning and faster convergence, we propose the improved LB-first Pulse algorithm by improving (decreasing) UB as early as possible. Specifically, we use the reverse shortest path trees to calculate the cost lower bounds LB_j of the partial path $\pi_p = (s, \dots, v)$ ending at node v to the destination t through the neighbor node j , where $LB_j = W^1(\pi_p) + w_{vj}^1 + W^1(\pi_j^1)$, and $(v, j) \in A$. All neighbors of π_p are ranked according to LB_j , and the neighbor with a smaller LB_j is more promising. Guided by this heuristic information, the neighbor j with a smaller LB_j will be preferred to be selected by the proposed LB-first search strategy as

Algorithm 3: Reductions and tests by infeasibility on resource upper and lower limits.

```

1: calculate the shortest path trees  $\bar{\pi}_t^k$  by performing Dijkstra's
   algorithm starting from  $s$  on  $G_k$ ;
   // Tests on upper limits
2: if  $W^k(\bar{\pi}_t^k) > U_k$  then
3:   Stop ( $\pi_*$  is the optimal path);
4: else if  $\bar{\pi}_t^k$  is feasible and  $W^1(\bar{\pi}_t^k) < UB$  then
5:    $UB \leftarrow W^1(\bar{\pi}_t^k)$ ,  $\pi_* \leftarrow \bar{\pi}_t^k$ ; ▷ improve UB
6: end if

   // Tests on lower limits
7: calculate the maximum estimate  $UB^k$  by Lemma 1;
8: if  $UB^k < L_k$  then
9:   Stop ( $\pi_*$  is the optimal path);
10: end if

11: calculate the reverse shortest path trees  $\bar{\pi}_t^k$  by performing Dijkstra's
   algorithm starting from  $t$  on  $G_k^1$ ;
   // Reductions on upper limits through nodes
12: for  $i \in V \setminus \{s, t\}$  do
13:   if  $W^k(\bar{\pi}_i^k) > U_k$  then
14:     delete node  $i$ ;
15:   end if
16: end for

   // Reductions on upper limits through arcs
17: for  $(i, j) \in A$  do
18:   if  $W^k(\bar{\pi}_{ij}^k) > U_k$  then
19:     delete arc  $(i, j)$ ;
20:   end if
21: end for

   // Reductions on lower limits through arcs
22: calculate the maximum estimate  $UB^k$  by Lemma 1;
23: if  $UB^k - \max_{(i,j) \in A} w_{ij}^k + \min_{(i,j) \in A} w_{ij}^k < L_k$  then
24:   for  $(i, j) \in A$  do
25:     calculate the maximum estimate  $UB_{ij}^k$  by Lemma 1;
26:     if  $UB_{ij}^k < L_k$  then
27:       delete arc  $(i, j)$ ;
28:     end if
29:   end for
30: end if

31: for  $(i, j) \in A$  do
32:   if  $\bar{\pi}_{ij}^k$  is feasible and  $W^1(\bar{\pi}_{ij}^k) < UB$  then
33:      $UB \leftarrow W^1(\bar{\pi}_{ij}^k)$ ,  $\pi_* \leftarrow \bar{\pi}_{ij}^k$ ; ▷ improve UB
34:   end if
35: end for

```

Algorithm 4: Final tests.

```

1: calculate the maximum estimate  $UB^k$  by the  $q$ -paths algorithm;
2: if  $UB^k < L_k$  for some  $k \in \{2, 3, \dots, K'\}$  then
3:   Stop ( $\pi_*$  is the optimal path);
4: end if

```

the successor node of π_p , which will usually yield better solution and improve UB earlier than other neighbor selection strategies, e.g., the uninformed depth-first search (DFS) of the Pulse algorithm.

The pseudocode for the algorithm in stage 2, the proposed LB-first Pulse algorithm with the initial cost upper bound UB from preprocessing, is given in Algorithm 5. The LB-first Pulse algorithm without the

Algorithm 5: LB-first Pulse algorithm.

Input: UB and the reverse shortest path trees $\bar{\pi}_i^k$ from the last iteration of preprocessing

Output: UB , π_* corresponding to UB

```

1: Init: current partial path  $\pi_p \leftarrow (s)$ ;
2: RECURSIVEPULSE( $s$ ,  $\pi_p$ );

3: function RECURSIVEPULSE(current node  $v$ , current partial path  $\pi_p$ )
4:    $J \leftarrow \{\}$ 
5:   for  $(v, j) \in A$  and  $j$  not in  $\pi_p$  (not visited) do
     // pruning by optimality on cost bounds
6:     if  $W^1(\pi_p) + w_{vj}^1 + W^1(\bar{\pi}_j^1) \geq UB$  then
7:       continue;
8:     end if
     // pruning by infeasibility on upper limits
9:     if  $W^k(\pi_p) + w_{vj}^k + W^k(\bar{\pi}_j^k) > U_k$  for some  $k \in \{2, 3, \dots, K'\}$ 
then
10:      continue;
11:     end if
12:     if  $v == t$  then
13:       if  $W^k(\pi_p) \geq L_k$  for all  $k \in \{2, 3, \dots, K\}$  then
14:          $UB \leftarrow W^1(\pi_p)$ ,  $\pi_* \leftarrow \pi_p$ ;
15:       end if
16:       continue;
17:     end if
18:      $LB_j = W^1(\pi_p) + w_{vj}^1 + W^1(\bar{\pi}_j^1)$ 
19:      $J = J \cup \{j\}$ 
20:   end for
21:   sort  $LB_j$  in increasing order;
22:   for  $j \in J$  corresponding to sorted  $LB_j$  do
23:      $\pi_p.add(j)$ ;
24:     RECURSIVEPULSE( $j$ ,  $\pi_p$ );
25:      $\pi_p.remove(j)$ ;
26:   end for
27: end function

```

preprocessing (initial UB is set to infinity) can also be used independently as a solution algorithm for the problem, which will be discussed later together with the two-stage method.

4. Computational experiments

In this section, we present the computational experiments on randomly generated instances with multiple double sided constraints to evaluate the performance of proposed algorithm for the DRCESPP. Our algorithms were implemented in Python and all experiments were run on a 64-bit Linux system configured with an 18-core Intel Core i9-7980XE CPU at 2.60 GHz and 128 GB of RAM. Each test instance used only one core, and parallelism was used only between instances. Note that all experimental results in this paper can be reproduced since the random seeds for each instance are saved. The source code is publicly available at <https://github.com/hellomuyi/DRCESPP>, and all original experimental data are listed in tabular form at the same link.

4.1. Test instances

Different from the commonly used ER random graph [38] as the topology of test instances, the more practical scale-free network based on the Barabási–Albert preferential attachment model [39] is adopted here. Its size is determined by the number of nodes $n = |V|$ and the number of edges to attach from a new node to existing nodes $\hat{m}$, and then the number of directed arcs will be $m = |A| = 2\hat{m}(n - \hat{m})$.

The feature (cost or resource consumption) values w_{ij}^k are uniformly distributed random numbers in the range (0, 1). For the $(k - 1)$ st resource, we define its resource lower limit $L_k = \frac{(1+q_1)W^k(\bar{\pi}_i^k) + (1-q_1)W^k(\bar{\pi}_i^1)}{2}$ and upper limit $U_k = \frac{(1-q_1)W^k(\bar{\pi}_i^k) + (1+q_1)W^k(\bar{\pi}_i^1)}{2}$, where $0 < q_1 < 1$ is the control parameter about constraint tightness, and lower values of q_1 indicate that the resource limits are tighter. For the cases that often appear in practice and are difficult to solve, another resource upper and lower limit setting method is proposed: $L_k = W^k(\bar{\pi}_i^1) + (W^k(\bar{\pi}_i^1) - W^k(\bar{\pi}_i^k))q_2 + \epsilon$ and $U_k = L_k + \frac{W^k(\bar{\pi}_i^1) - W^k(\bar{\pi}_i^k)}{10} - 2\epsilon$, where $-1 < q_2 < 1$ is the control parameter about interval position, and ϵ is a small positive number for clear interval boundaries. The upper and lower limits determined by $q_2 \approx 0$ are generally larger and tighter than that determined by q_1 , as is often the case in practice.

To summarize, each instance type is determined by the number of features K , the number of nodes $|V|$, the number of edges $|A|$, and the resource limit parameter q_1 or q_2 . Two classes of test instances are studied. The instances in class 1, constrained by q_1 , are generated with $|K| \in \{2, 5\}$, $|V| \in \{100, 500, 1000\}$, $\hat{m} \in |V|^{\{0.4, 0.6, 0.8\}}$ ($|A|$ ranges from 1128 to 375 998) and $q_1 \in \{0.8, 0.6, 0.4, 0.2, 0.1\}$, leading to a total of 90 instances types. While for the instances constrained by q_2 in class 2, we adopt $|K| \in \{2, 5\}$, $|V| \in \{100, 500\}$, $\hat{m} \in |V|^{\{0.6, 0.7, 0.8\}}$ ($|A|$ ranges from 2550 to 102 528) and $q_2 \in \{-0.2, -0.1, 0.0, 0.1\}$, and a total of 48 types are generated.

4.2. Experiments for preprocessing algorithm

To evaluate the performance of the preprocessing algorithm w.r.t. the proposed maximum resource consumption estimation method, we designed three algorithms for comparison: the preprocessing without maximum resource consumption estimation methods (without reductions/tests by infeasibility on resource lower limits), the preprocessing using only the maximum estimate from Lemma 1, and the preprocessing using maximum estimates from both Lemma 1 and the q -paths algorithm, labeled as Est0, Est1, and Est2, respectively. Est2 is the complete version of our Algorithm 1, Est1 is its version without using line 23 of Algorithm 1, and Est0 is the version without the estimation techniques for the resource lower limits.

The repeat-until loop (line 22 of Algorithm 1) of preprocessing will not be broken unless falling into a deadlock, meaning that the maximum number of iterations will not take effect in the experiments in this paper. There are 1000 randomly generated instances with different random seeds for each instance type (parameter combination).

Comparative experiments were conducted on the instances in class 1 and class 2. Two evaluation metrics, the average process time and the number of solved instances in preprocessing, were chosen, and the results in class 1 and class 2 are shown in Figs. 1 and 2, respectively. As Fig. 1 shows, for clarity, the 90 averaged result items of baseline (Est0) in class 1 are sorted in ascending order by their evaluation metrics (result items of Est1 and Est2 are repositioned accordingly) and divided equally into 6 groups to display the means within each group. Results in class 2 do the same.

Our findings are listed below.

1. The effectiveness of the maximum estimation method is demonstrated by comparing the number of instances solved by preprocessing. Compared with Est0, Est1 can solve much more instances without significant additional time overhead. And Est2 further solved more instances than Est1, but the used final test is computationally costly for the tightly constrained instances.
2. The value of preprocessing is that it cannot only make the problem more likely to be solved to optimality in the first stage, but if not, it can eliminate a large number of edges and the resulting initial cost bounds can accelerate the algorithm solution in the second stage.

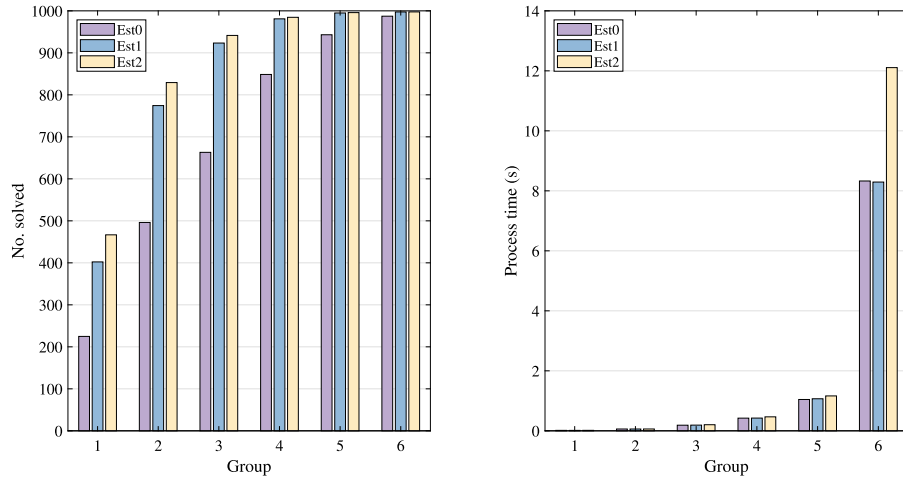


Fig. 1. Number of solved instances in preprocessing and average process time on instances in class 1.

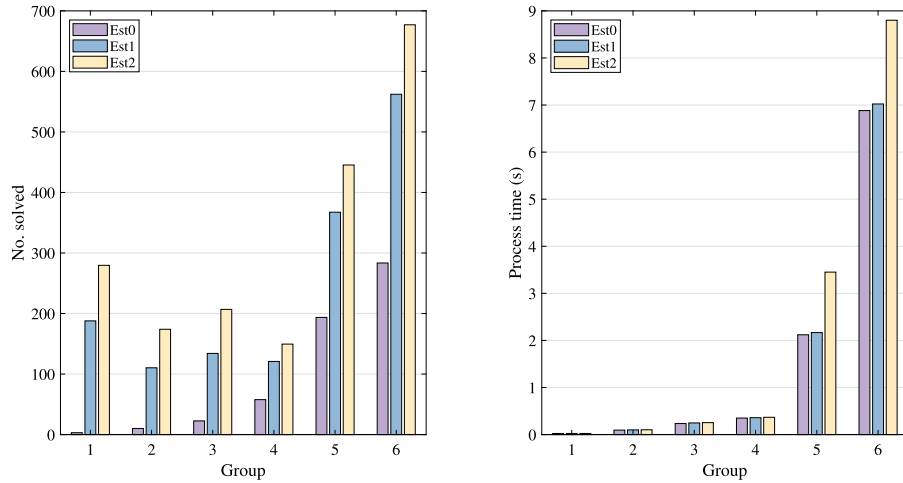


Fig. 2. Number of solved instances in preprocessing and average process time on instances in class 2.

- For many instance types, e.g., $q_1 \geq 0.4$, most instances (almost over 90%) were solved directly by preprocessing in the first stage. Furthermore, preprocessing generally yielded feasible solutions and eliminated a large number of edges for the remaining unsolved instances. Therefore, only preprocessing is sufficient for such simple cases if the exact solution is not demanding.
- When multiple resources or tight resource limits (low values of q_1) are considered, the problem is difficult to solve, and the advantage of preprocessing will be limited, possibly due to the difficulty of finding feasible solutions and improving cost upper bounds. However, for these challenging instances, the value of preprocessing is still not negligible. For instance, a reduction of 67.69% for the number of arcs and a ratio of 780/834 about cost bounds benefiting the second stage algorithm can be obtained by Est2 (Algorithm 1) for the instances with $K = 5$, $q_1 = 0.1$, $|V| = 1000$ and $|A| = 375\,998$.

The above conclusions are verified through the results in class 2. Besides, it can be found that the proposed maximum resource consumption estimation method had significant advantages for the instances with single resource ($K = 2$). In particular, for $K = 2$ and $q_2 = 0.0$ or 0.1 , Est1 and Est2 could solve a considerable proportion of all 1000 instances, whereas Est0 solved almost no instances.

4.3. Experiments for the LB-first Pulse algorithm and two-stage method

The performance of the LB-first Pulse algorithm (Algorithm 5) and the two-stage method (Algorithm 1 + Algorithm 5) is evaluated by conducting experiments on difficult instances in class 2. We generate 100 instances for each of the 48 instance types. The results are also sorted and divided, and shown in Fig. 3. We can find from the solution time that both the LB-first Pulse Algorithm and the two-stage method are computationally more efficient than the Pulse algorithm, with speedups ranging from 1.52 to 15.93 and from 1.38 to 76.06 respectively, especially for larger-scale dense networks. In most cases, the two-stage method is more efficient than the pure LB-first Pulse algorithm and achieves higher speedups. Besides, it is very difficult for Pulse algorithm to solve the instances with tight and large resource limits, since the available pruning by bounds and infeasibility is difficult to take effect. And the performance of the Pulse algorithm deteriorates sharply with increasing values of q_2 (larger upper and lower limits), while the LB-first Pulse algorithm and the two-stage method behave relatively mildly and robustly, and this is clear for the case with multiple resources.

5. Conclusions and future research

In this paper, we focus on exploring solution methods for the doubly resource constrained elementary shortest path problem (DRCESPP) and

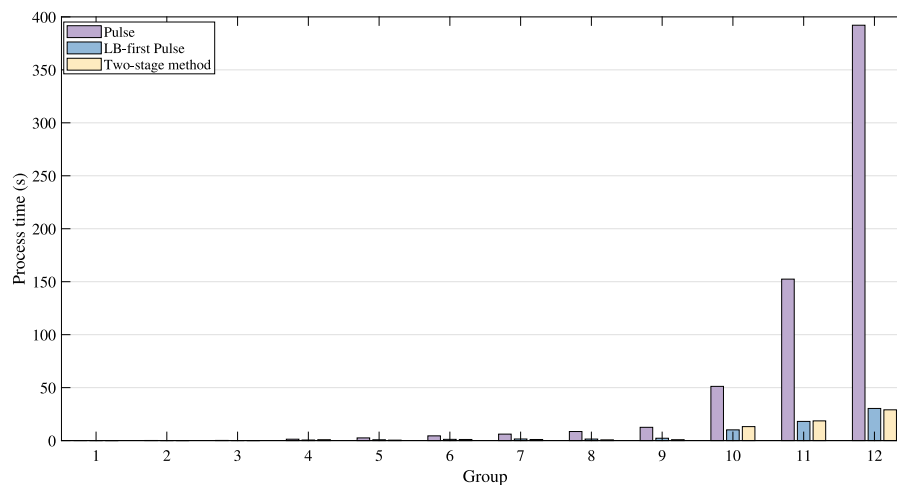


Fig. 3. Average process time on instances in class 2.

the problem has multiple double sided resource constraints. The main difficulty lies in the need for specialized techniques for resource lower limit constraints. To address this, we propose a method to estimate the maximum resource consumption, and the extensive experiments demonstrated the effectiveness of the method. That is, the proposed preprocessing can significantly reduce the network size without incurring too much time overhead. Besides, we propose the improved LB-first Pulse algorithm with a more effective search strategy instead of the original uninformed search. Additional experiments on instances with tight and large resource limits show that the LB-first Pulse algorithm and the two-stage method have significant advantages over the Pulse algorithm in terms of solution time, with speedups ranging from 1.52 to 15.93 and from 1.38 to 76.06 respectively. This paper enriches and extends the solution method of DRCSPP, and the proposed method will serve practical application scenarios such as routing computation and differential service.

For future research, more effective maximum resource consumption estimation methods can be explored to accelerate the preprocessing and even the second-stage algorithms. In addition, attention can be paid to finding more efficient (exact or heuristic) algorithms to replace the slightly more computationally expensive algorithms in stage 2 for the difficult instances with multiple resources and tight and large resource limits. For instance, combining deep learning, especially deep reinforcement learning, is a promising direction [40,41].

CRediT authorship contribution statement

Xuefeng Jin: Writing – review & editing, Visualization, Validation, Software, Methodology, Data curation, Conceptualization. **Shunzheng Yu:** Writing – original draft, Supervision, Software, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] M. Minoux, C. Ribeiro, A transformation of hard (equality constrained) knapsack problems into constrained shortest path problems, *Oper. Res. Lett.* 3 (4) (1984) 211–214, [http://dx.doi.org/10.1016/0167-6377\(84\)90028-2](http://dx.doi.org/10.1016/0167-6377(84)90028-2).
- [2] M.R. Garey, D.S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W. H. Freeman, 1979.
- [3] M. Dror, Note on the complexity of the shortest path models for column generation in VRPTW, *Oper. Res.* 42 (5) (1994) 977–978, <http://dx.doi.org/10.1287/opre.42.5.977>.
- [4] C.C. Ribeiro, M. Minoux, A heuristic approach to hard constrained shortest path problems, *Discrete Appl. Math.* 10 (2) (1985) 125–137, [http://dx.doi.org/10.1016/0166-218X\(85\)90007-1](http://dx.doi.org/10.1016/0166-218X(85)90007-1).
- [5] J.E. Beasley, N. Christofides, An algorithm for the resource constrained shortest path problem, *Networks* 19 (4) (1989) 379–394, <http://dx.doi.org/10.1002/net.3230190402>.
- [6] D.H. Wolpert, W.G. Macready, *No Free Lunch Theorems for Search*, Working Papers 95-02-010, Santa Fe Institute, 1995.
- [7] D.H. Wolpert, The lack of a priori distinctions between learning algorithms, *Neural Comput.* 8 (7) (1996) 1341–1390, <http://dx.doi.org/10.1162/neco.1996.8.7.1341>.
- [8] G.Y. Handler, I. Zang, A dual algorithm for the constrained shortest path problem, *Networks* 10 (4) (1980) 293–309, <http://dx.doi.org/10.1002/net.3230100403>.
- [9] K. Mehlhorn, M. Ziegelmann, Resource constrained shortest paths, in: G. Goos, J. Hartmanis, J. van Leeuwen, M.S. Paterson (Eds.), *Algorithms - ESA 2000*, Vol. 1879, Springer Berlin Heidelberg, Berlin, Heidelberg, 2000, pp. 326–337, http://dx.doi.org/10.1007/3-540-45253-2_30.
- [10] W.M. Carlyle, J.O. Royset, R. Kevin Wood, Lagrangian relaxation and enumeration for solving constrained shortest-path problems, *Networks* 52 (4) (2008) 256–270, <http://dx.doi.org/10.1002/net.20247>.
- [11] R. Muhandirange, N. Boland, Simultaneous solution of Lagrangean dual problems interleaved with preprocessing for the weight constrained shortest path problem, *Networks* 53 (4) (2009) 358–381, <http://dx.doi.org/10.1002/net.20292>.
- [12] G. Feng, T. Korkmaz, Finding multi-constrained multiple shortest paths, *IEEE Trans. Comput.* 64 (9) (2015) 2559–2572, <http://dx.doi.org/10.1109/TC.2014.2366762>.
- [13] Y.P. Aneja, V. Aggarwal, K.P.K. Nair, Shortest chain subject to side constraints, *Networks* 13 (2) (1983) 295–302, <http://dx.doi.org/10.1002/net.3230130212>.
- [14] I. Dumitrescu, N. Boland, Improved preprocessing, labeling and scaling algorithms for the weight-constrained shortest path problem, *Networks* 42 (3) (2003) 135–153, <http://dx.doi.org/10.1002/net.10090>.
- [15] D. Feillet, P. Dejax, M. Gendreau, C. Gueguen, An exact algorithm for the elementary shortest path problem with resource constraints: Application to some vehicle routing problems, *Networks* 44 (3) (2004) 216–229, <http://dx.doi.org/10.1002/net.20033>.
- [16] G. Righini, M. Salani, Symmetry helps: Bounded bi-directional dynamic programming for the elementary shortest path problem with resource constraints, *Discrete Optim.* 3 (3) (2006) 255–273, <http://dx.doi.org/10.1016/j.disopt.2006.05.007>.
- [17] G. Righini, M. Salani, New dynamic programming algorithms for the resource constrained elementary shortest path problem, *Networks* 51 (3) (2008) 155–170, <http://dx.doi.org/10.1002/net.20212>.
- [18] X. Zhu, W.E. Wilhelm, A three-stage approach for the resource-constrained shortest path as a sub-problem in column generation, *Comput. Oper. Res.* 39 (2) (2012) 164–178, <http://dx.doi.org/10.1016/j.cor.2011.03.008>.

- [19] L. Lozano, A.L. Medaglia, On an exact method for the constrained shortest path problem, *Comput. Oper. Res.* 40 (1) (2013) 378–384, <http://dx.doi.org/10.1016/j.cor.2012.07.008>.
- [20] L. Lozano, D. Duque, A.L. Medaglia, An exact algorithm for the elementary shortest path problem with resource constraints, *Transp. Sci.* 50 (1) (2016) 348–357, <http://dx.doi.org/10.1287/trsc.2014.0582>.
- [21] N. Cabrera, A.L. Medaglia, L. Lozano, D. Duque, An exact bidirectional pulse algorithm for the constrained shortest path, *Networks* 76 (2) (2020) 128–146, <http://dx.doi.org/10.1002/net.21960>.
- [22] J. Li, X. Han, Revised pulse algorithm for elementary shortest path problem with resource constraints, in: 2019 Seventh International Symposium on Computing and Networking, CANDAR, IEEE, Nagasaki, Japan, 2019, pp. 37–44, <http://dx.doi.org/10.1109/CANDAR.2019.00013>.
- [23] J.E. Kelley, The cutting-plane method for solving convex programs, *J. Soc. Ind. Appl. Math.* 8 (4) (1960) 703–712, <http://dx.doi.org/10.1137/0108053>, [arXiv:2099058](https://arxiv.org/abs/2099058).
- [24] J.Y. Yen, Finding the K shortest loopless paths in a network, *Manage. Sci.* 17 (11) (1971) 712–716, <http://dx.doi.org/10.1287/mnsc.17.11.712>.
- [25] B.L. Fox, D.M. Landi, Searching for the multiplier in one-constraint optimization problems, *Oper. Res.* 18 (2) (1970) 253–262, <http://dx.doi.org/10.1287/opre.18.2.253>.
- [26] W. Matthew Carlyle, R. Kevin Wood, Near-shortest and K -shortest simple paths, *Networks* 46 (2) (2005) 98–109, <http://dx.doi.org/10.1002/net.20077>.
- [27] L. Santos, J. Coutinho-Rodrigues, J.R. Current, An improved solution algorithm for the constrained shortest path problem, *Transp. Res. B* 41 (7) (2007) 756–771, <http://dx.doi.org/10.1016/j.trb.2006.12.001>.
- [28] A. Sedeño-Noda, S. Alonso-Rodríguez, An enhanced K -SP algorithm with pruning strategies to solve the constrained shortest path problem, *Appl. Math. Comput.* 265 (2015) 602–618, <http://dx.doi.org/10.1016/j.amc.2015.05.109>.
- [29] B.W. Thomas, T. Calogiuri, M. Hewitt, An exact bidirectional A^* approach for solving resource-constrained shortest path problems, *Networks* 73 (2) (2019) 187–205, <http://dx.doi.org/10.1002/net.21856>.
- [30] S. Ahmadi, G. Tack, D.D. Harabor, P. Kilby, A fast exact algorithm for the resource constrained shortest path problem, *Proc. AAAI Conf. Artif. Intell.* 35 (14) (2021) 12217–12224, <http://dx.doi.org/10.1609/aaai.v35i14.17450>.
- [31] N. Kohl, Exact Methods for Time Constrained Routing and Related Scheduling Problems (Ph.D. thesis), Technical University of Denmark, 1995.
- [32] N. Boland, J. Dethridge, I. Dumitrescu, Accelerated label setting algorithms for the elementary resource constrained shortest path problem, *Oper. Res. Lett.* 34 (1) (2006) 58–68, <http://dx.doi.org/10.1016/j.orl.2004.11.011>.
- [33] A.B. Kahn, Topological sorting of large networks, *Commun. ACM* 5 (11) (1962) 558–562, <http://dx.doi.org/10.1145/368996.369025>.
- [34] R. Saigal, Letter to the editor—a constrained shortest route problem, *Oper. Res.* 16 (1) (1968) 205–209, <http://dx.doi.org/10.1287/opre.16.1.205>.
- [35] M. Rosseel, Letter to the editor—comments on a paper by romesh saigal: “A constrained shortest route problem”, *Oper. Res.* 16 (6) (1968) 1232–1234, <http://dx.doi.org/10.1287/opre.16.6.1232>.
- [36] R. Bellman, On a routing problem, *Quart. Appl. Math.* 16 (1) (1958) 87–90, <http://dx.doi.org/10.1090/qam/102435>.
- [37] L.R. Ford, Network flow theory, 1956.
- [38] P. Erdos, A. Rényi, et al., On the evolution of random graphs, *Publ. Math. Inst. Hung. Acad. Sci.* 5 (1) (1960) 17–60.
- [39] A.-L. Barabási, R. Albert, Emergence of scaling in random networks, *Science* 286 (5439) (1999) 509–512, <http://dx.doi.org/10.1126/science.286.5439.509>.
- [40] Y. Bengio, A. Lodi, A. Prouvost, Machine learning for combinatorial optimization: A methodological tour d’horizon, *European J. Oper. Res.* 290 (2) (2021) 405–421, <http://dx.doi.org/10.1016/j.ejor.2020.07.063>.
- [41] C. Zhang, Y. Wu, Y. Ma, W. Song, Z. Le, Z. Cao, J. Zhang, A review on learning to solve combinatorial optimisation problems in manufacturing, *IET Collaborat. Intell. Manuf.* 5 (1) (2023) e12072, <http://dx.doi.org/10.1049/cim2.12072>.